응용 8_4_1 파일

코드 업그레이드

발표자 : 노정균	2021963025

목차

1	응용 8_4_1 파일의 구조	파일의 구조가 어떻게 되어있지?
2	파일 업그레이드	기존 코드에서 뭐가 추가되었지?
3	업그레이드 총 정리	업그레이드 내역이 어떻게 되지?

응용 8_4_1 파일의 구조 (1)

```
#include <stdio.h>
#include <conio.h>
#include <windows.h>
void gotoxy(int x, int y);
void display_text(int count);
void display_question(void);
void display_result(void);
void intro_game(void);
char sol[10][11];
int main(void) {
    intro_game();
    display_question();
    printf("\n문제풀이가 모두 끝났습니다.\n");
    printf("\n\n풀이를 보려면 아무키나 누르시오.");
    getch();
    display_result();
    return 0;
}
```

[전체 흐름]

- ① 시작 안내
- ② 질문 전체 진행
- ③ 키 입력 대기
- ④ 해설 전체 진행
- ⑤ 프로그램 종료

응용 8_4_1 파일의 구조 (2)

```
void gotoxy(int x, int y)
{
    COORD Pos = {x - 1, y - 1};
    SetConsoleCursorPosition(
        GetStdHandle(STD_OUTPUT_HANDLE), Pos);
}
```

COORD Pos : Pos 라는 자료형 변수에 콘솔의 좌표를 담음

x - 1, y - 1을 하는 이유 : 사용자는 (1,1) 부터 입력 받기 때문이다.

GetStdHandle(STD_OUTPUT_HANDLE), Pos) : 해당 좌표 위치에 출력 핸들을 놓음

응용 8_4_1 파일의 구조 (3)

```
void display_text(int count) {
    gotoxy(1,2);
    printf("다음 문제를 읽고 답을 입력하시오.\n\n");
    printf("(%d번 문제) \n\n", count+1);
}
```

1. 화면을 초기화 한다.
2. printf를 모두 실행하여 문자를 출력한다.
3. 사용자가 키를 하나 누를 때까지 기다립니다. 키를 누르면 함수가 끝나고 다음 단계로 진행합니다.

응용 8_4_1 파일의 구조 (4)

```
void display_question(void) {
    FILE *fp1;
    char chr1[82];
    int line = 5, count = 0;
    char *file1 = "심리테스트질문.txt";

  fp1 = fopen(file1, "r");
  system("cls");

  while (!feof(fp1)) {
    fgets(chr1, 80, fp1);

    if (chr1[0] == 10) {
      display_text(count);
      gotoxy(1, line);
      scanf("%s", sol[count]);
      system("cls");
      line = 5;
      count++;
      }
```

```
    else {
      gotoxy(1, line++);
      printf("%s", chr1);
      }
  }
  display_text(count);
  gotoxy(1, line);
  scanf("%s", sol[count]);
  fclose(fp1);
}
```

1. 질문 파일을 연다.
2. 파일에서 한 줄씩 읽는다.
3. 빈 줄을 만나면 답을 입력받아 sol[count]에 저장한다.
4. 파일 끝에 도달하면 마지막 문제의 답을 입력받는다.
5. 파일을 닫는다

응용 8_4_1 파일의 구조 (5)

```
void display_result(void) {
 FILE *fp2;
 char chr1[82];

 int line=5, count=0;
 char *file2="심리테스트해설.txt";
 fp2=fopen(file2, "r");
 system("cls");

 while(!feof(fp2)) {
       fgets(chr1, 80, fp2);

       if (chr1[0]==10) {
             gotoxy(1, line+1);
             printf("당신의 답은 : %s\n", sol[count]);
             printf("다음문제의 해설은 아무키나 누르시오.");
             getch();
             system("cls");
             line=5;
             count++;
        }
```

```
else {
       gotoxy(1, line++);
       printf("%s", chr1);
        }
  }
 gotoxy(1, line+1);
 printf("당신의 답은 : %s\n", sol[count]);
 fclose(fp2);
}
```

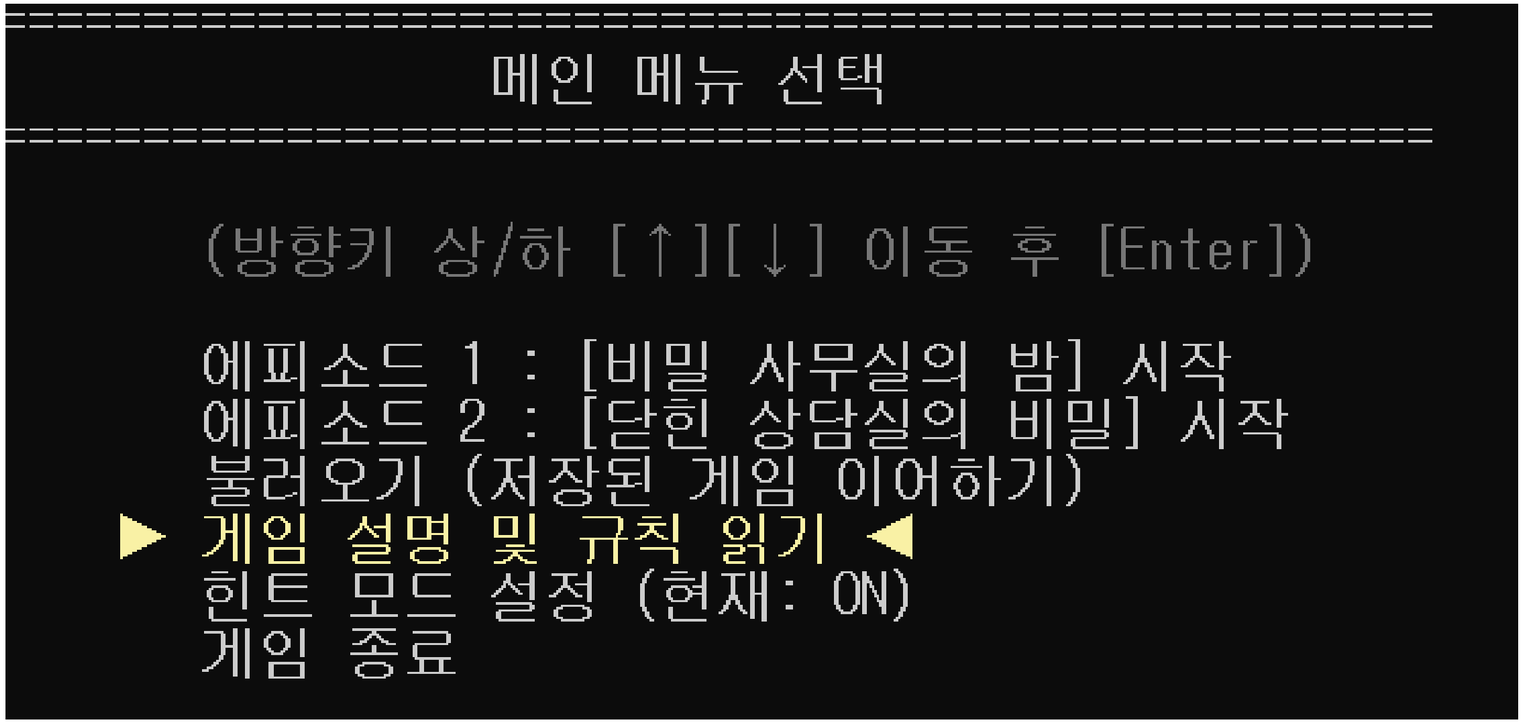
1. 해설 파일을 연다.
2. 파일을 한 줄씩 읽어 화면에 출력한다.
   - ㄴ 읽은 줄이 빈 줄이 아니면 현재 행에 해설 내용을 출력
   - ㄴ 빈 줄을 만나면, 현재 순서에 해당하는 답을 함께 출력
3. 키 입력을 기다린 뒤 화면을 지우고 다음 해설을 준비한다.
4. 반복문이 끝나면 마지막 답을 출력하고 파일을 닫는다.

응용 8_4_1 파일의 구조 (6)

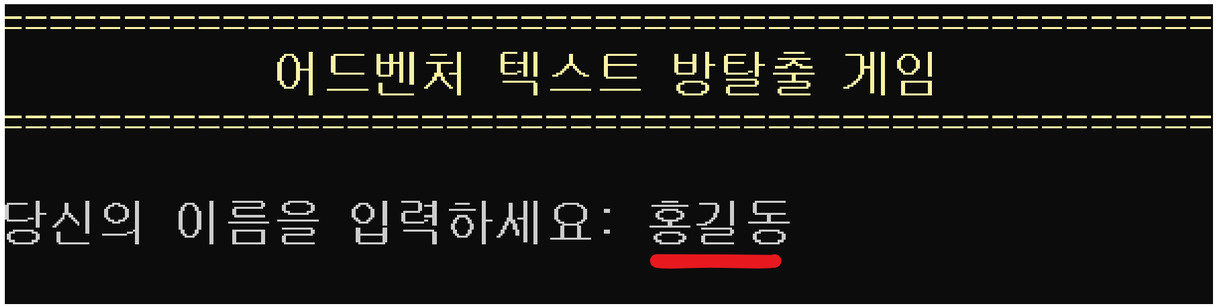
```
void intro_game(void) {
    system("cls");
    printf("심리테스트 게임\n\n");
    printf("제시되는 문제에 모두 답을 하고나면\n");
    printf("해설이 출력됩니다.\n\n");
    printf("아무키나 누르면 시작합니다. ");
    getch();
}
```

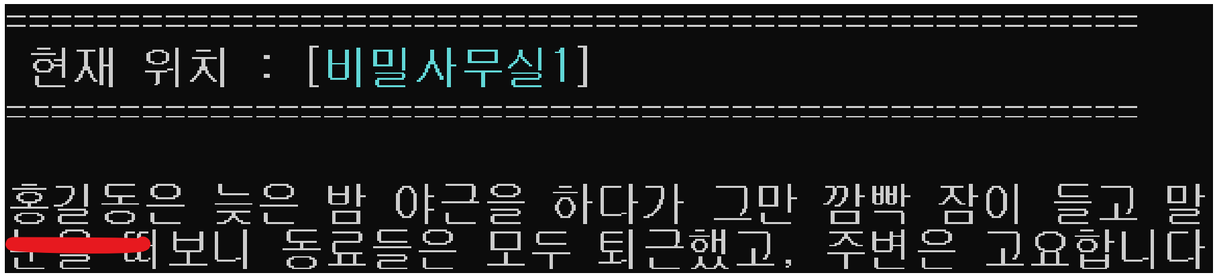
1. count를 전달받는다.
2. 안내 문구와 문제 번호(count + 1)를 출력한다.
3. 아무 문자를 입력해 함수를 종료시킨다.

파일 업그레이드


메인 메뉴 선택
(방향키 상/하 [↑][↓] 이동 후 [Enter])
에피소드 1 : [비밀 사무실의 밤] 시작
에피소드 2 : [닫힌 상담실의 비밀] 시작
불러오기 (저장된 게임 이어하기)
▶ 게임 설명 및 규칙 읽기 ◀
힌트 모드 설정 (현재: ON)
게임 종료

파일 업그레이드 (1)





[사용자 닉넴 입력과 활용]

```c
void intro_game(void) {
    // ... (콘솔 크기 및 폰트 설정 코드 생략) ...

    printf("플레이어 이름을 입력하세요: ");
    scanf("%49s", player_name);  // 띄어쓰기 전까지의 문자열을 player_name 변수에
    clear_input_buffer();        // 엔터(개행 문자) 등 입력 버퍼 비우기

    // 치트키 기능: 입력받은 이름이 "관리자"인지 확인
    if (strcmp(player_name, "관리자") == 0) cheat_mode = 1;
    else cheat_mode = 0;
}
```

파일 업그레이드 (2)

[HINT 모드 - OFF]

```
홍길동은 늦은 밤 야근을 하다가 그만 깜빡 잠이 들고 말았습니다.
눈을 떠보니 동료들은 모두 퇴근했고, 주변은 고요합니다.
퇴근을 하려고 나서보지만, 출입구 쪽에는 굳게 닫힌 도어락이 보입니다.

홍길동의 의자에는 입고 온 외투가 걸려 있고,
어질러진 책상 위에는 결재 서류와 작은 메모장이 놓여 있습니다.
벽면에는 커다란 달력이 하나 걸려 있고,
문 옆에는 사원증을 태그하는 단말기가 붉은빛을 깜빡이고 있습니다.

(어떻게든 이곳을 빠져나가야 한다. 단서가 될 만한 것을 조사하자.)
```

[HINT 모드 - ON]

```
홍길동은 늦은 밤 야근을 하다가 그만 깜빡 잠이 들고 말았습니다.
눈을 떠보니 동료들은 모두 퇴근했고, 주변은 고요합니다.
퇴근을 하려고 나서보지만, 출입구 쪽에는 굳게 닫힌 [도어락]이 보입니다.

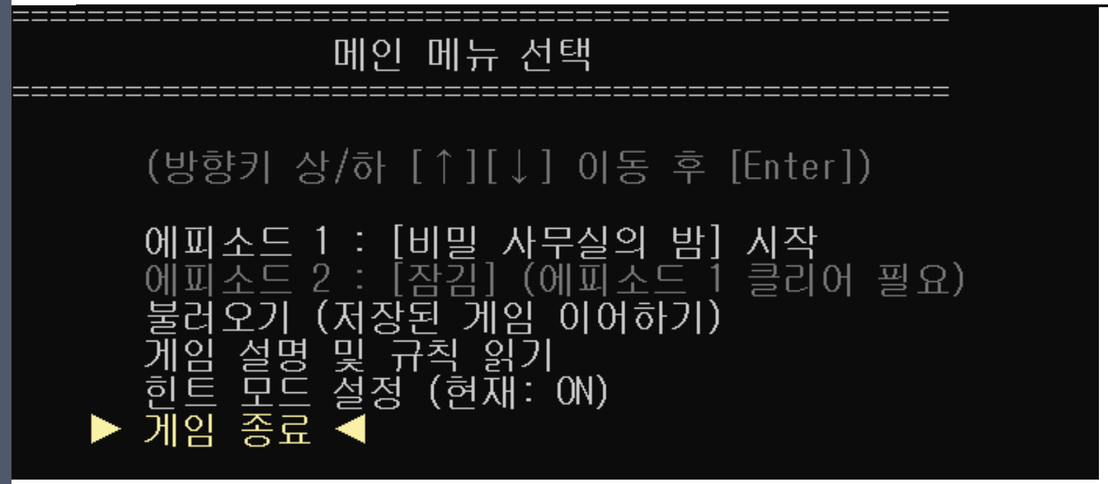
홍길동의 의자에는 입고 온 [외투]가 걸려 있고,
어질러진 [책상] 위에는 결재 서류와 작은 [메모장]이 놓여 있습니다.
벽면에는 커다란 [달력]이 하나 걸려 있고,
문 옆에는 사원증을 태그하는 [단말기]가 붉은빛을 깜빡이고 있습니다.

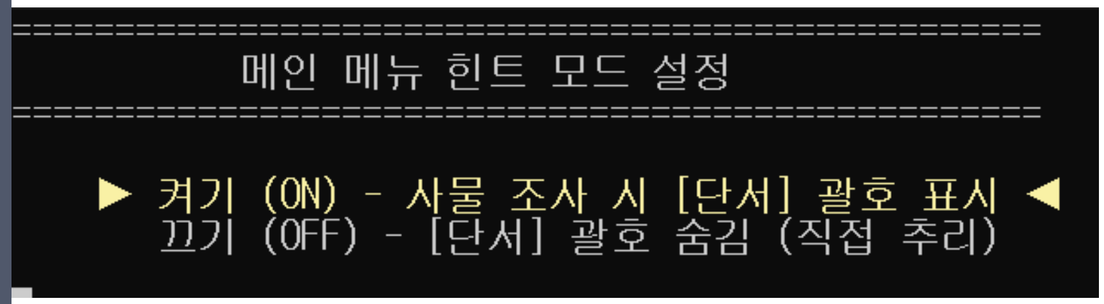
(어떻게든 이곳을 빠져나가야 한다. 단서가 될 만한 것을 조사하자.)
```

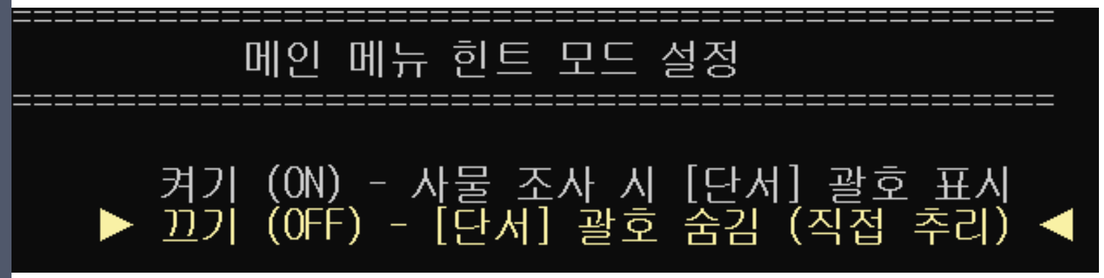
[HINT 모드]

```c
if (difficulty_mode == 2 || (difficulty_mode == 3 && hint_mode == 1)) {
    printf("%s", temp); // 대괄호가 포함된 원본 문장 그대로 출력
}
// HARD, NORMAL 이거나 힌트가 꺼져있다면?
else {
    // 문자열을 한 글자씩 검사함
    for (int i = 0; temp[i] != '\0'; i++) {
        // 만약 현재 글자가 '[' 이거나 ']' 가 아니면 화면에 출력 (즉, 괄호 글자만 스킵함)
        if (temp[i] != '[' && temp[i] != ']') putchar(temp[i]);
    }
}
```

파일 업그레이드 (3)







[메인 메뉴 선택과 HINT 모드(방향키 상하 조작)]

```
key = getch(); // 사용자의 키 입력을 받음

// 1. 방향키(특수키)가 눌렸는지 확인 (방향키는 224 또는 0을 먼저 반환함)
if (key == 224 || key == 0) {
    key = getch(); // 실제 방향키가 무엇인지 두 번째 값을 읽어옴

    // ⬆️ 위쪽 방향키 (아스키코드 72)
    if (key == 72) {
        choice--; // 선택 인덱스를 1 줄임 (위로 이동)
        if (choice < 0) choice = 5; // 맨 위에서 한 번 더 올리면 맨 아래(5)로 이동
        sound_navigate(); // 이동 효과음 재생
    }
    // ⬇️ 아래쪽 방향키 (아스키코드 80)
    else if (key == 80) {
        choice++; // 선택 인덱스를 1 늘림 (아래로 이동)
        if (choice > 5) choice = 0; // 맨 아래에서 한 번 더 내리면 맨 위(0)로 이동
        sound_navigate(); // 이동 효과음 재생
    }
}
// 2. 엔터키(Enter)가 눌렸는지 확인 (아스키코드 13)
else if (key == 13) {
    // ... (선택한 메뉴 실행 로직) ...
    sound_action();
    return choice; // 선택한 메뉴의 인덱스 번호를 반환하고 종료
}
```

파일 업그레이드 (4)

[무의미한 행동의 이벤트 처리]

```
문 옆에는 사원증을 태그하는 [단말기]가 붉은빛을 깜빡이고 있습니다.

(어떻게든 이곳을 빠져나가야 한다. 단서가 될 만한 것을 조사하자.)

------------------------------------------------
[ 명령어: 인벤토리, 조합, 분해, 힌트, 사물/장소 이름 ]
▶ 행동 입력 (0: 나가기, 9: 저장 후 나가기): 붉은빛
```

```
>> '붉은빛' - 없는 행동이거나, 여긴 없습니다.

[ ESC 키를 누르면 돌아갑니다... ]
```

```c
if (is_event) {
    wait_for_esc();
}
// 2차: 이벤트 단어가 아니라면 파일이 있는지 확인
else {
    // "현재방_입력한단어.txt" 형식으로 파일명을 만듦 (예: "신방실1_창문.txt")
    sprintf(obj_file, "%s_%s.txt", room_names[current_room], user_answer);
    FILE* fp = fopen(obj_file, "r");

    // 파일이 존재한다면 (잘된 행동 - 단순 조사)
    if (fp != NULL) {
        // ... (텍스트 파일 내용 읽어서 출력하는 로직 생략) ...
    }
    // ❌ 파일이 존재하지 않는다면 (잘못된 행동!)
    else {
        system("cls");
        set_color(C_ERROR); // 빨간색으로 경고 표시

        // 유저가 입력한 단어를 그대로 보여주며 없는 행동임을 안내
        printf("\n>> '%s' - 없는 행동이거나, 아무것도 없습니다.\n", user_answer);

        set_color(C_DEFAULT);
        sound_error(); // 삐-빅! 하는 에러음 발생
        wait_for_esc();
    }
}
```

파일 업그레이드 (5)

```
홍길동은 늦은 밤 야근을 하다가 그만 깜빡 잠이 들고 말았습니다.
눈을 떠보니 동료들은 모두 퇴근했고, 주변은 고요합니다.
퇴근을 하려고 나서보지만, 출입구 쪽에는 굳게 닫힌 [도어락]이 보입니다.

홍길동의 의자에는 입고 온 [외투]가 걸려 있고,
어질러진 [책상] 위에는 결재 서류와 작은 [메모장]이 놓여 있습니다.
벽면에는 커다란 [달력]이 하나 걸려 있고,
문 옆에는 사원증을 태그하는 [단말기]가 붉은빛을 깜빡이고 있습니다.

(어떻게든 이곳을 빠져나가야 한다. 단서가 될 만한 것을 조사하자.)

------------------------------------------------------
[ 명령어: 인벤토리, 조합, 분해, 힌트, 사물/장소 이름 ]
▶ 행동 입력 (0: 나가기, 9: 저장 후 나가기): 외투
```

```
>> 안주머니에서 [사원증] 획득!

[ ESC 키를 누르면 돌아갑니다... ]
```

```
>> 외투 주머니는 텅 비어있다.

[ ESC 키를 누르면 돌아갑니다... ]
```

[의미있는 행동의 이벤트 처리]

```
if (strcmp(user_answer, "서랍") == 0) {
    system("cls"); // 화면을 한 번 지워줌

    // 1 사원증을 아직 얻지 않은 상태일 때 (!has_id_card)
    if (!has_id_card) {
        set_color(C_ITEM); // 아이템 획득 색상(노란색)으로 변경
        printf("\n>> 서랍주머니에서 [사원증] 획득!\n");
        set_color(C_DEFAULT);

        sound_item();    // 띠리링~ 하는 아이템 획득 효과음 재생
        has_id_card = 1; // 💡 핵심: 사원증을 가지고 있다는 상태(1)로 변경하여 게임이 기억하게 함
    }
```

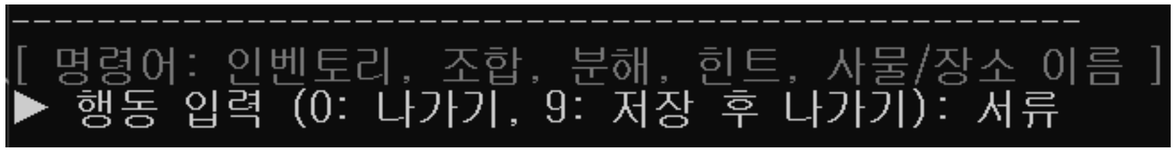
[의미있는 행동의 중복 방지 이벤트 처리]

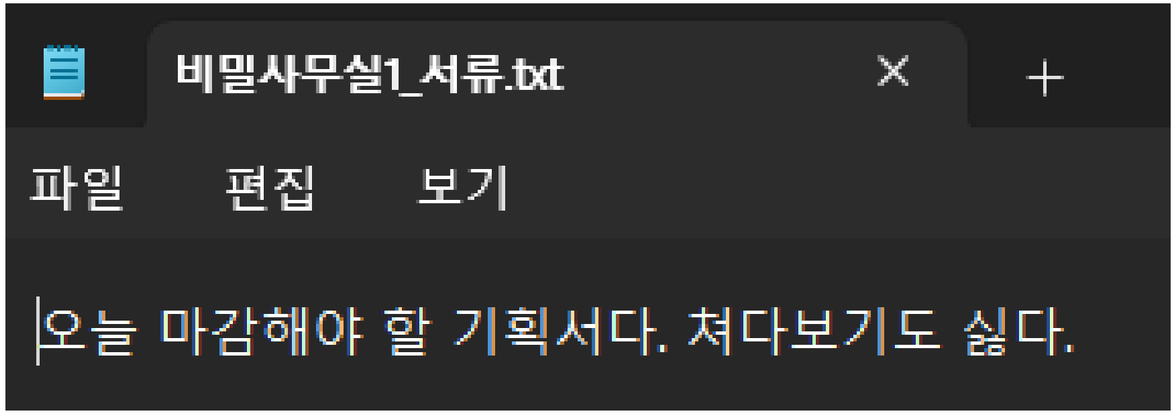
```
    else {
        set_color(C_INFO); // 안내 색상(하늘색)으로 변경
        printf("\n>> 서랍 주머니는 텅 비어있다.\n");
        set_color(C_DEFAULT);

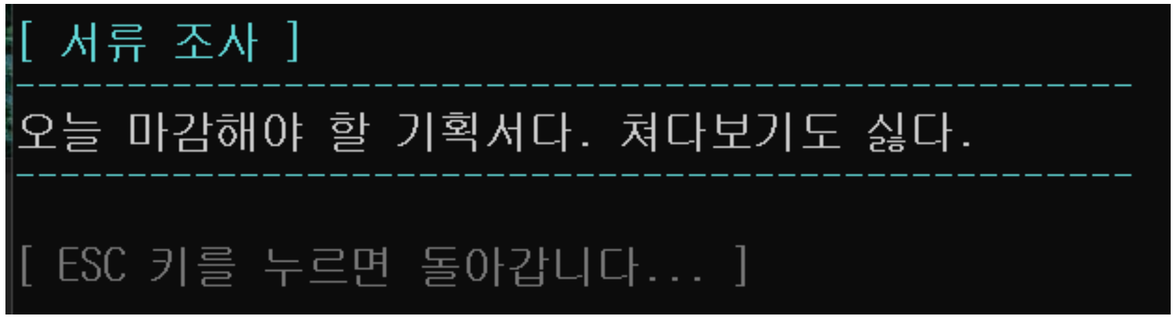
        sound_action();  // 띡- 하는 일반 조사 효과음 재생
    }

    is_event = 1; // 이벤트로 처리되었음을 체크 (없는 행동으로 처리되지 않도록 방지)
}
```

파일 업그레이드 (6)







[의미있는 행동의 파일 읽기 처리]

```c
while (fgets(line, sizeof(line), fp) != NULL) {
    char temp[1024] = {0};
    char *pos = line;
    char *match;

    // 🔍 가공 1: "이름"이라는 단어를 찾아서 플레이어의 실제 이름으로 바꿔줌
    while ((match = strstr(pos, "이름")) != NULL) {
        strncat(temp, pos, match - pos);
        strcat(temp, player_name);
        pos = match + strlen("이름");
    }
    strcat(temp, pos);

    // 🔍 가공 2: 난이도에 따라 대괄호 [ ] 숨기기
    if (difficulty_mode == 2 || (difficulty_mode == 3 && hint_mode == 1)) {
        printf("%s", temp); // EASY 모드: 괄호까지 그대로 출력
    } else {
        for (int k = 0; temp[k] != '\0'; k++) {
            if (temp[k] != '[' && temp[k] != ']') putchar(temp[k]); // 그 외: 괄호 빼
        }
    }
```

파일 업그레이드 (7)

```
>> 문이 굳게 잠겨있다. 옆에 있는 [단말기]를 조작해야 한다.

[ ESC 키를 누르면 돌아갑니다... ]
```

```
>> 카드를 태그해야 한다. [사원증]이 필요하다.
```

```
>> 안주머니에서 [사원증] 획득!
```

```
>> 사원증 인식.

[ 보안 시스템 단말기 ]
오늘 날짜(TODAY)를 8자리로 입력하라. (취소: ESC)
```

```
>> 확인 완료. 철컥! 출입구 도어락 잠금이 해제되었습니다!
```

```
>> 단말기를 통해 도어락이 해제되어 있다. [복도]로 갈 수 있다.
```

[흐름 이벤트 트리거]

```
else if (strcmp(user_answer, "출입문") == 0 || strcmp(user_answer, "문") == 0) {
    system("cls");
    if (is_door_unlocked) { /* 잠금 해제 시 로직 */ }
    else {
        // 🔒 문이 잠겨있는 상태
        set_color(C_ERROR);
        printf("\n>> 굳게 잠겨있다. 옆에 있는 [단말기]를 조작해야 한다.\n");
        set_color(C_DEFAULT);
        sound_error();
    }
    is_event = 1;
```

```
else if (strcmp(user_answer, "단말기") == 0) {
    system("cls");
    if (!has_id_card) {
        // 🚫 사원증이 없는 상태
        set_color(C_ERROR);
        printf("\n>> 카드를 태그해야 한다. [사원증]이 필요하다.\n");
        set_color(C_DEFAULT);
        sound_error();
```

파일 업그레이드 (8)

[흐름 HINT 1]

```
==================================================
                [ 흐름 힌트 ]
==================================================

>> 외투나 책상 등 주변을 샅샅이 조사하여 단말기를 켤 물건을 찾으세요.

[ ESC 키를 누르면 돌아갑니다... ]
```

[흐름 이벤트 트리거 1 완성]

```
>> 안주머니에서 [사원증] 획득!

[ ESC 키를 누르면 돌아갑니다... ]
```

[흐름 HINT 2]

```
==================================================
                [ 흐름 힌트 ]
==================================================

>> [사원증]을 얻었으니 [단말기]를 조작해보세요. 비밀번호는 달력과 메모장을 참고!

[ ESC 키를 누르면 돌아갑니다... ]
```

[흐름 HINT]

```
if (episode == 1) {
    if (current_room == 1) {

        // 1 [ 흐름 HINT 1 ] : 아직 사원증을 얻지 못했을 때
        if (!has_id_card) {
            printf(">> 책상과 책꽂이 주변을 자세히 조사하여 단말기를 켤 방법을 찾아보세요.\n");
        }

        // 2 [ 흐름 HINT 2 ] : 사원증은 얻었지만, 단말기(문)는 풀지 않았을 때
        // (위의 if문이 거짓이 되고, 여기가 참이 됨)
        else if (!is_door_unlocked) {
            printf(">> [사원증]을 획득했으니 [단말기]를 작동해보세요. 비밀번호는 달력과 메모지를 참
        }

        // 3 [ 흐름 HINT 3 ] : 단말기까지 모두 풀어서 문이 열렸을 때
        else {
            printf(">> 문이 열렸습니다. [출입문]을 통해 [복도]로 이동하세요.\n");
        }

    }
    // ... (이후 2번 방, 3번 방에 대한 힌트 로직 이어짐)
```

파일 업그레이드 (9)

```
==================================================
                [ 아이템 조합 ]
==================================================

[★] 현재 가진 아이템 목록:
 (가진 아이템이 없습니다)

--------------------------------------------------
[ESC] 취소하고 돌아가기 / [Enter] 조합 시작
```

```
==================================================
                [ 흐름 힌트 ]
==================================================

>> 외투나 책상 등 주변을 샅샅이 조사하여 단말기를

[ ESC 키를 누르면 돌아갑니다... ]
```

```
>> 안주머니에서 [사원증] 획득!

[ ESC 키를 누르면 돌아갑니다... ]
```

[esc 입력으로 전 화면으로 돌아가는 로직]

```
else if (strcmp(user_answer, "조합") == 0) {
    // ... (화면 지우고 아이템 목록 띄우는 코드 생략) ...

    printf("\n--------------------------------------------------\n");
    // 💡 이미지에 밑줄 친 바로 그 안내문!
    printf(" [ESC] 취소하고 돌아가기 / [Enter] 조합 시작\n");

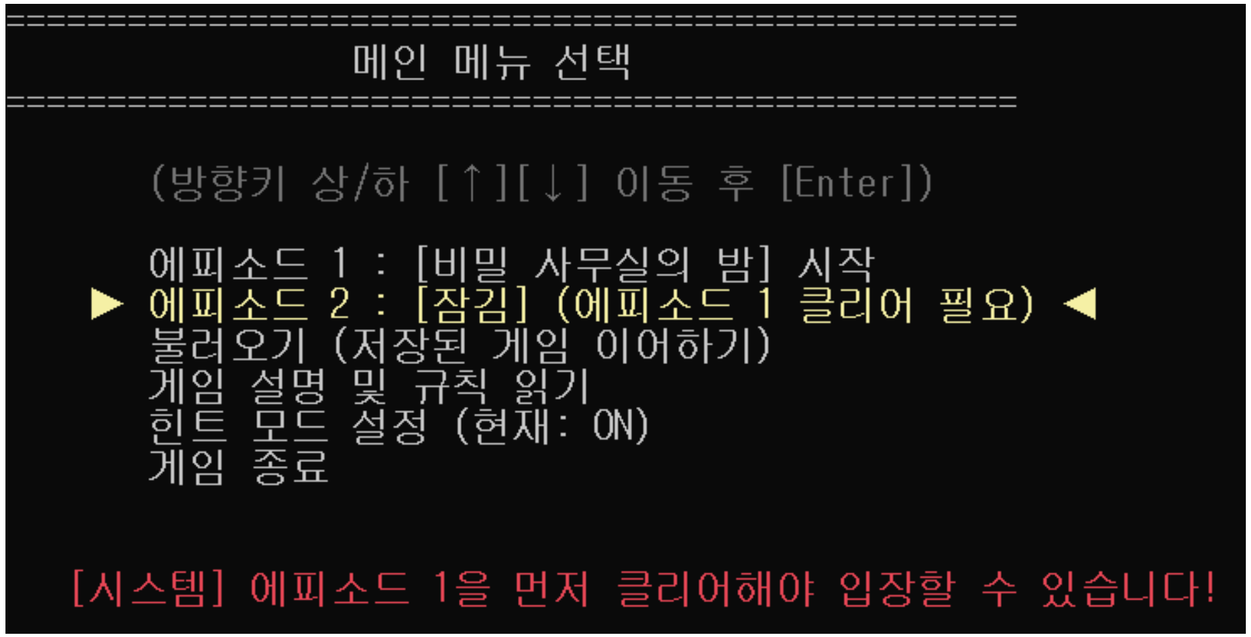
    int key;
    // 무한 루프를 돌면서 키보드 입력을 기다림
    while(1) {
        key = getch();
        if (key == 27 || key == 13) break; // 27(ESC)이나 13(Enter)을 누르면 대기 중단
    }

    // 💡 만약 누른 키가 27(ESC)이라면?
    if (key == 27) continue; // 아이템을 입력받지 않고 상위 방 화면(루프 처음)으로 돌아감!
```

```
else if (strcmp(user_answer, "힌트") == 0) {
    // ... (난이도 검사 및 현재 방에 맞는 힌트 문구 출력 코드 생략) ...

    // 💡 힌트 출력이 모두 끝나면, 만들어둔 도우미 함수를 호출하여 화면을 멈춥니다!
    wait_for_esc();
    continue; // 화면이 풀리면 다시 행동을 입력받는 곳으로 돌아감
}
```

파일 업그레이드 (10)



[에피소드 해금 시스템]

```
int is_cleared = play_episode(1, 0); // 에피소드 1 플레이 시작

// 💡 탈출에 성공해서 1(클리어)을 반환받았다면?
if (is_cleared == 1) {
    ep1_cleared = 1;   // 클리어 플래그를 1로 변경! (이제 에피소드
    save_clear_data(); // 변경된 플래그를 파일(clear.dat)에 저장하
```

파일 업그레이드 (11)

[조합 기능]

[조합 가능 이벤트 처리]

```
  - 사원증
  - 건전지
  - 고장난손전등

------------------------------------------------
 [ESC] 취소하고 돌아가기 / [Enter] 조합 시작

첫 번째 템 (취소: 0): 건전지
두 번째 템 (취소: 0): 고장난손전등

>> 건전지와 고장난손전등을 조합!
>> [손전등]을 만들었습니다!

[ ESC 키를 누르면 돌아갑니다... ]
```

```c
// 💡 입력 순서에 상관없이 (A+B 또는 B+A) 조합되도록 OR(||) 연산자 사용!
if ((strcmp(item1, "건전지") == 0 && strcmp(item2, "망가진손전등") == 0) ||
    (strcmp(item1, "망가진손전등") == 0 && strcmp(item2, "건전지") == 0))
{
    // 유저가 실제로 그 두 아이템을 모두 가지고 있는지 2차 검증!
    if (has_battery && has_broken_flash) {

        set_color(C_ITEM);
        printf("\n>> 건전지와 망가진손전등을 결합했다!\n>> [손전등]을 완성했습니다!\n");
        set_color(C_DEFAULT);
        sound_item(); // 뽀로롱~ 아이템 획득음

        // 💡 핵심: 재료 아이템은 없애고(0), 완성된 아이템은 생성(1)함!
        has_battery = 0;
        has_broken_flash = 0;
        has_flash = 1;
    }
    else {
        // 이름은 맞췄지만 실제로 안 가지고 있을 때의 예외 처리 (아래에서 설명)
    }
```

파일 업그레이드 (12)

[조합 기능]

[조합 불가능 이벤트 처리]

```
[★] 현재 가진 아이템 목록:
 - 사원증
 - 건전지
 - 고장난손전등

------------------------------------------------
 [ESC] 취소하고 돌아가기 / [Enter] 조합 시작

첫 번째 템 (취소: 0): 사원증
두 번째 템 (취소: 0): 건전지

>> 서로 조합할 수 없거나 없는 템입니다.
```

```c
// 💡 입력 순서에 상관없이 (A+B 또는 B+A) 조합되도록 OR(||) 연산자 사용!
if ((strcmp(item1, "건전지") == 0 && strcmp(item2, "망가진손전등") == 0) ||
    (strcmp(item1, "망가진손전등") == 0 && strcmp(item2, "건전지") == 0))
{
    // 유저가 실제로 그 두 아이템을 모두 가지고 있는지 2차 검증!
    if (has_battery && has_broken_flash) {

        set_color(C_ITEM);
        printf("\n>> 건전지와 망가진손전등을 결합했다!\n>> [손전등]을 완성했습니다!\n");
        set_color(C_DEFAULT);
        sound_item(); // 뽀로롱~ 아이템 획득음

        // 💡 핵심: 재료 아이템은 없애고(0), 완성된 아이템은 생성(1)함!
        has_battery = 0;
        has_broken_flash = 0;
        has_flash = 1;
    }
    else {
        // 이름은 맞췄지만 실제로 안 가지고 있을 때의 예외 처리 (아래에서 설명)
    }
```

파일 업그레이드 (13)

```
==================================================
                [ 아이템 분해 ]
==================================================

 [★] 현재 가진 아이템 목록:
  - 사원증
  - 손전등

--------------------------------------------------
 [ESC] 취소하고 돌아가기 / [Enter] 분해 시작

분해할 아이템 이름 입력 (취소: 0)
▶ 손전등

>> 손전등을 분해했습니다!
>> [건전지]와 [고장난손전등]을 다시 얻었습니다.

[ ESC 키를 누르면 돌아갑니다... ]
```

[분해 기능]

[분해 가능 이벤트 처리]

```c
// 유저가 분해할 아이템으로 "손전등"을 입력했을 때
if (strcmp(d_item, "손전등") == 0) {

    // 유저가 실제로 [손전등]을 가지고 있는지 확인!
    if (has_flash) {
        set_color(C_ITEM);
        printf("\n>> 손전등을 분해했습니다!\n");
        printf(">> [건전지]와 [망가진손전등]을 다시 획득했습니다.\n");
        set_color(C_DEFAULT);
        sound_item(); // 뽀로롱~ 아이템 획득음

        // 💡 핵심: 완성품(손전등) 플래그는 끄고(0), 재료들(건전지, 망가진손전등)은 다시 켬(1)!
        has_flash = 0;
        has_battery = 1;
        has_broken_flash = 1;
    }
```

파일 업그레이드 (14)

```
==================================================
                [ 아이템 분해 ]
==================================================

 [★] 현재 가진 아이템 목록:
  - 사원증
  - 손전등

--------------------------------------------------
 [ESC] 취소하고 돌아가기 / [Enter] 분해 시작

분해할 아이템 이름 입력 (취소: 0)
▶ 사원증

>> 분해할 수 없는 물건입니다.

[ ESC 키를 누르면 돌아갑니다... ]
```

[분해 기능]

[분해 불가능 이벤트 처리]

```
    else {
        // ❌ 아이템 미소지 에러
        set_color(C_ERROR);
        printf("\n>> 해당 아이템을 가지고 있지 않습니다.\n");
        set_color(C_DEFAULT);
        sound_error(); // 삐-빅! 에러음
    }
```

```
else {
    set_color(C_ERROR);
    printf("\n>> 분해할 수 없는 물건입니다.\n");
    set_color(C_DEFAULT);
    sound_error();
}

wait_for_esc(); // 유저가 에러 문구를 읽고 ESC를 누르면 방으로 돌아감
continue;
```

파일 업그레이드 (15)

```
------------------------------------------------
[ 명령어: 인벤토리, 조합, 분해, 힌트, 사물/장소 이름 ]
▶ 행동 입력 (0: 나가기, 9: 저장 후 나가기): 인벤토리
```

```
================================================
                [ 인벤토리 ]
================================================

- 사원증
- 건전지 (AAA 사이즈)
- 고장난손전등

[ ESC 키를 누르면 돌아갑니다... ]
```

[인벤토리 확인 기능]

```c
// 플레이어가 "인벤토리"라고 입력했을 때
if (strcmp(user_answer, "인벤토리") == 0) {
    system("cls"); // 화면 지우기
    printf("================================================\n");
    printf("                  [ 인벤토리 ]                  \n");
    printf("================================================\n\n");

    set_color(C_ITEM); // 아이템 전용 색상(노란색)으로 글자색 변경

    // 💡 empty(비어있음) 변수를 1(참)로 시작
    int empty = 1;

    // 각 아이템의 소지 여부(플래그)가 1(참)인지 개별적으로 검사하여 출력
    if(has_id_card) { printf("- 사원증\n"); empty = 0; }
    if(has_battery) { printf("- 건전지 (AAA 건전지)\n"); empty = 0; }
    if(has_broken_flash) { printf("- 망가진손전등\n"); empty = 0; }
    if(has_flash) { printf("- 손전등 (작동 가능)\n"); empty = 0; }
    if(has_key1) { printf("- 작은 철제 열쇠\n"); empty = 0; }
    if(has_remote) { printf("- 리모컨 (건전지가 없음)\n"); empty = 0; }
    if(has_working_remote) { printf("- 작동하는리모컨\n"); empty = 0; }
    if(has_golden_key) { printf("- 황금열쇠 (금속상자 열쇠)\n"); empty = 0; }
    if(has_usb) { printf("- 보안USB (비밀번호 단서)\n"); empty = 0; }
    if(has_masterkey) { printf("- 마스터키\n"); empty = 0; }

    // 💡 위에서 단 하나라도 아이템을 가져서 empty가 0이 되지 않았다면?
    if(empty) printf("  (현재 소지품이 없습니다)\n");
```

파일 업그레이드 (16)

```
[★] 현재 가진 아이템 목록:
  - 사원증
  - 건전지
  - 고장난손전등

------------------------------------------------
 [ESC] 취소하고 돌아가기 / [Enter] 조합 시작

첫 번째 템 (취소: 0): 사원증
두 번째 템 (취소: 0): 건전지

>> 서로 조합할 수 없거나 없는 템입니다.
```

```
================================================
                [ 아이템 분해 ]
================================================

 [★] 현재 가진 아이템 목록:
  - 사원증
  - 손전등

------------------------------------------------
 [ESC] 취소하고 돌아가기 / [Enter] 분해 시작

분해할 아이템 이름 입력 (취소: 0)
▶ 손전등

>> 손전등을 분해했습니다!
>> [건전지]와 [고장난손전등]을 다시 얻었습니다.

[ ESC 키를 누르면 돌아갑니다 ]
```

[특수한 경우 인벤토리 확인 기능]

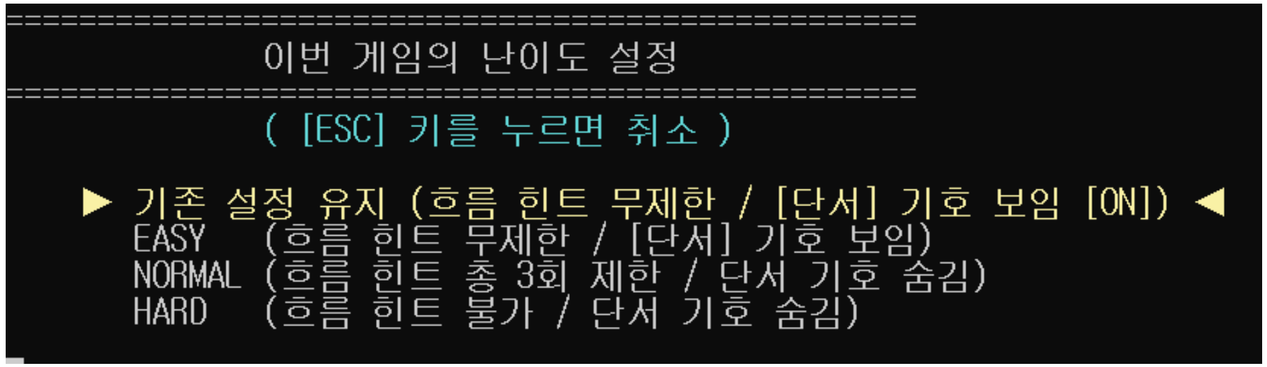
```c
printf(" [★] 현재 가진 아이템 목록:\n");
set_color(C_ITEM); // 노란색으로 변경

int empty = 1; // 비어있는지 확인하는 플래그

// 가지고 있는(1) 아이템들만 화면에 출력!
if(has_id_card) { printf("  - 사원증\n"); empty = 0; }
if(has_battery) { printf("  - 건전지\n"); empty = 0; }
if(has_broken_flash) { printf("  - 망가진손전등\n"); empty = 0; }
if(has_flash) { printf("  - 손전등\n"); empty = 0; }
if(has_key1) { printf("  - 작은 철제 열쇠\n"); empty = 0; }
if(has_remote) { printf("  - 리모컨\n"); empty = 0; }
if(has_working_remote) { printf("  - 작동하는리모컨\n"); empty = 0; }
if(has_golden_key) { printf("  - 황금열쇠\n"); empty = 0; }
if(has_usb) { printf("  - 보안USB\n"); empty = 0; }
if(has_masterkey) { printf("  - 마스터키\n"); empty = 0; }

// 가진 게 하나도 없다면 안내문 출력
if(empty) printf("  (가진 아이템이 없습니다)\n");
set_color(C_DEFAULT); // 다시 기본 색상으로 복구
```

파일 업그레이드 (17)



[난이도 설정 기능]

```c
else if (strcmp(user_answer, "힌트") == 0) {

    // 🔴 HARD 모드: 힌트 기능 자체를 차단
    if (difficulty_mode == 0) {
        set_color(C_ERROR);
        printf(">> [!] HARD 모드에서는 진행 힌트를 사용할 수 없습니다.\n");
        set_color(C_DEFAULT);
        sound_error();
    }
    // 🟠 NORMAL 모드: 3회까지만 허용하고 이후 차단
    else if (difficulty_mode == 1 && flow_hint_count >= 3) {
        set_color(C_ERROR);
        printf(">> [!] NORMAL 모드의 진행 힌트를 모두 소진했습니다. (3/3)\n");
        printf(">> 이제 스스로의 힘으로 일어서야 합니다!\n");
        set_color(C_DEFAULT);
        sound_error();
    }
    // 🟢 EASY / NORMAL(횟수 남음) / 기본 모드: 힌트 제공
    else {
        // NORMAL 모드일 경우에만 힌트 카운트를 1씩 증가시키고 남은 횟수 안내
        if (difficulty_mode == 1) {
            flow_hint_count++;
            printf(">> [안내] 남은 진행 힌트 횟수 : %d회 (총 3회)\n\n", 3 - flow_hint_count);
        }
```

파일 업그레이드 (18)

```
>> 챕터 소요 시간 : 0분 45초
>> 챕터 획득 랭크 : [ S ] 등급
```

[소요 시간]

```c
void animate_clear_screen(int episode, int final_time) {
    // ... (화면 콩팡거리는 클리어 연출 로직 생략) ...

    gotoxy(0, 12); // 커서 위치 이동
    printf(" >> 탈출 소요 시간 : %d분 %d초\n", final_time / 60, final_time % 60);
```

[랭크 기능]

```c
// seconds: 플레이어가 클리어하는 데 걸린 총 시간(초)
// is_total: 개별 에피소드인지(0), 1+2 합산 전체 시간인지(1) 판별하는 플래그
char get_rank(int seconds, int is_total) {

    // 💡 핵심: 합산 시간(is_total == 1)이라면 랭크 커트라인을 2배로 늘려줌!
    int multiplier = is_total ? 2 : 1;

    if (seconds <= 120 * multiplier) return 'S';      // 2분(합산 4분) 이내 : S
    else if (seconds <= 240 * multiplier) return 'A'; // 4분(합산 8분) 이내 : A
    else if (seconds <= 420 * multiplier) return 'B'; // 7분(합산 14분) 이내 : B
    else if (seconds <= 720 * multiplier) return 'C'; // 12분(합산 24분) 이내 : C
    else if (seconds <= 1200 * multiplier) return 'D';// 20분(합산 40분) 이내 : D
    else return 'F';                                  // 그 이상 걸리면 : F
}
```

파일 업그레이드 (19)

```
==================================================
 [ 에피소드 1 클리어 ] 마침내 모든 층을 빠져나왔습니다!
==================================================
```

```
==================================================
 [ 에피소드 1 클리어 ] 마침내 모든 층을 빠져나왔습니다!
==================================================

 >> 챕터 소요 시간 : 0분 36초
 >> 챕터 획득 랭크 : [ S ] 등급


[ ESC 키를 누르면 돌아갑니다... ]
```

[동적 클리어 씬 화면]

```c
void animate_clear_screen(int episode, int final_time) {
    // ...
    // 15번의 프레임(장면)을 반복해서 그려냅니다.
    for(int i = 0; i < 15; i++) {

        // 1 좌표 이동 (Slide Down 효과)
        int y_offset = i;
        if (y_offset > 8) y_offset = 8; // 너무 밑으로 안 내려가게 Y좌표 제한

        // 2 색상 점멸 (Flashing 효과)
        // 프레임(i)이 바뀔 때마다 초록(10) -> 노랑(14) -> 하늘색(11)으로 번갈아가며 색 변경
        int color = (i % 3 == 0) ? 10 : ((i % 3 == 1) ? 14 : 11);
        set_color(color);

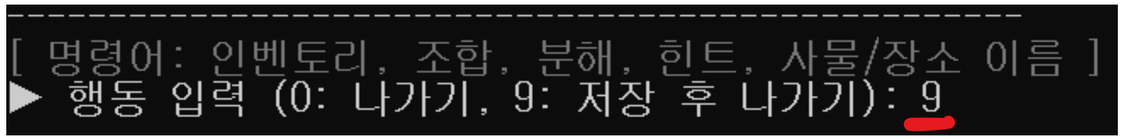
        // 3 화면에 그리기
        gotoxy(0, y_offset); // 계산된 위치로 커서 이동
        printf("==================================================\n");
        printf(" [ 에피소드 클리어 ] 축하합니다! \n");
        printf("==================================================\n");

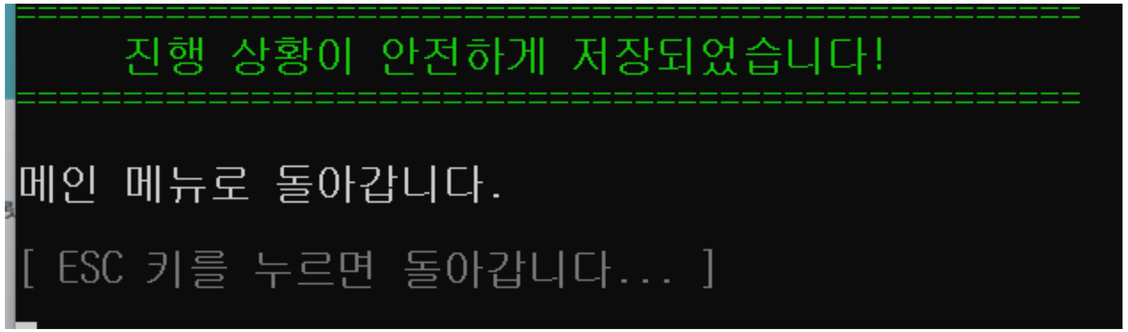
        // 4 프레임 딜레이 (애니메이션 속도 조절)
        Sleep(120); // 0.12초 동안 화면을 멈춰서 사람 눈에 보이게 함

        // 5 이전 프레임 지우기 (잔상 제거)
        if (i < 14) {
            gotoxy(0, y_offset);
            printf("                                                  \n");
            // 빈칸을 덮어씌워서 방금 그린 글자를 지움!
        }
    }
```

파일 업그레이드 (20)

[저장 및 이어하기 기능]






```c
if (strcmp(user_answer, "9") == 0 || strcmp(user_answer, "저장") == 0) {
    // "save.dat" 파일을 쓰기(w) 모드로 생성 또는 덮어쓰기
    FILE* fp = fopen("save.dat", "w");

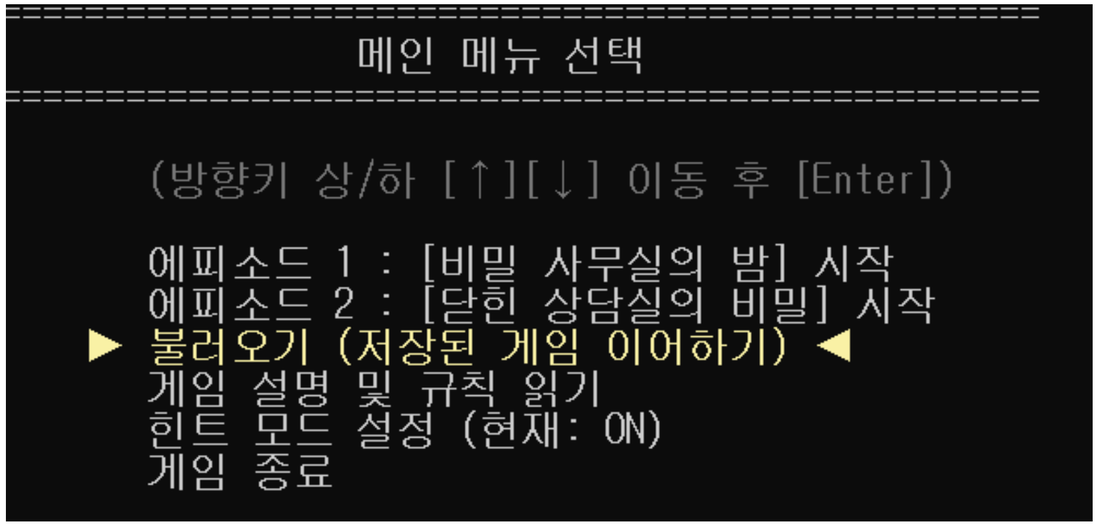
    if (fp != NULL) {
        // 💡 핵심: 현재까지 플레이한 '누적 시간'을 계산
        int current_acc = acc_time + (int)difftime(time(NULL), start_time);

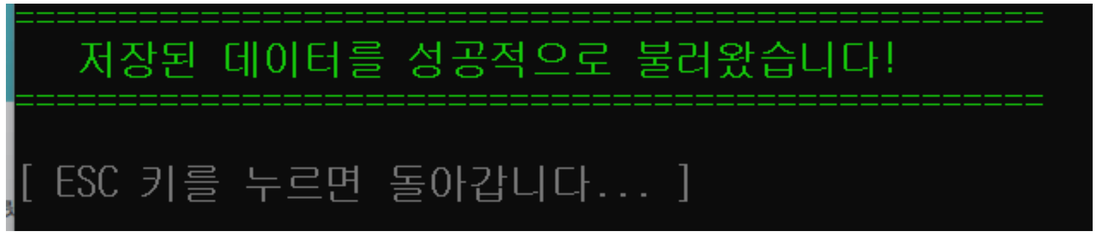
        // 게임의 모든 변수(총 19개)를 띄어쓰기로 구분하여 파일에 한 줄로 쭉 적어 넣습니다!
        fprintf(fp, "%d %d %d %d %d %d %d %d %d %d %d %d %d %d %d %d %d %d %d",
            episode, current_room, is_door_unlocked,                // 1. 에피소드, 현재 방 위
            has_id_card, has_battery, has_broken_flash, has_flash, has_key1, // 2. 인벤토리
            has_remote, has_working_remote, has_golden_key, has_usb, has_masterkey, // 2.
            current_acc, ep1_time, ep2_time,                        // 3. 플레이 시간 정보
            difficulty_mode, flow_hint_count, hint_mode);           // 4. 난이도 및 힌트 설정

        fclose(fp); // 파일 닫기 (저장 완료)

        // (저장 성공 UI 메시지 출력 후 메인 메뉴로 돌아감)
        return 0;
    }
}
```

파일 업그레이드 (21)





[저장 및 이어하기 기능]

```
else if (choice == 2) {
    // "save.dat" 파일을 읽기(r) 모드로 엽니다.
    FILE* fp = fopen("save.dat", "r");

    // ❌ 만약 파일이 없다면? (저장한 적이 없거나 삭제됨)
    if (fp == NULL) {
        system("cls");
        set_color(C_ERROR);
        printf(">> 저장된 데이터가 없습니다!\n"); // 에러 출력 후 메뉴로 복귀
        set_color(C_DEFAULT);
        sound_error();
        wait_for_esc();
    }
    // ⭕ 파일이 존재한다면? (저장된 데이터 있음)
    else {
        int saved_episode;
        // 파일의 첫 번째 숫자(저장해 둔 에피소드 번호)만 먼저 슬쩍 읽어옵니다.
        fscanf(fp, "%d", &saved_episode);
        fclose(fp);

        // 💡 알아낸 에피소드 번호를 넣고, 두 번째 인자(is_load)를 1(불러오기 모드)로 세팅해서 게
        int clear_loaded = play_episode(saved_episode, 1);

        // ... (클리어 후 처리 로직)
    }
}
```

파일 업그레이드 (22)

```
==================================================
                  메인 메뉴 선택
==================================================

      (방향키 상/하 [↑][↓] 이동 후 [Enter])

      에피소드 1 : [비밀 사무실의 밤] 시작
      에피소드 2 : [닫힌 상담실의 비밀] 시작
      불러오기 (저장된 게임 이어하기)
    ▶ 게임 설명 및 규칙 읽기 ◀
      힌트 모드 설정 (현재: ON)
      게임 종료
```

```
--------------------------------------------------
                  [ 게임 설명 ]
==================================================

1. 방 안의 묘사를 읽고 의심 가는 '사물'을 조사하세요.
2. [ 난이도 시스템 ] (에피소드 시작 시 선택)
 - 기존 설정 : 흐름 힌트 무제한 + 괄호 표시는 메인메뉴 ON/OFF
연동
 - EASY : 단서에 [ ] 표시. 흐름 힌트 무제한!
 - NORMAL : 괄호 숨김. 흐름 힌트 3회 제한.
 - HARD : 괄호 숨김 & 흐름 힌트 없음! 순수 실력으로 탈출.

3. [특수 명령어]
 - 인벤토리 / 조합 / 분해 / 힌트
 - 0 : 그냥 나가기 / 9 : 현재 진행 상황 저장하고 나가기

4. 단서를 본 뒤엔 [ESC] 키로 돌아오세요.
```

[게임 설명 옵션 기능 추가]

```c
void show_rules(void) {
    system("cls"); // 기존 메뉴 화면을 지우고 깨끗한 화면으로 만듦

    printf("==================================================\n");
    printf("                  [ 게임 설명 ]                   \n");
    printf("==================================================\n\n");

    // 1. 기본 조작법 설명
    printf("1. 각 방의 지문을 읽고 의심 가는 사물 '이름'을 입력하세요.\n");

    // 2. 난이도 시스템 설명
    printf("2. [ 난이도 시스템 ] (에피소드 진입 전 선택)\n");
    printf(" - 기본 설정 : 진행 힌트 무제한 + 힌트 표시용 대괄호 ON/OFF 가능\n");
    printf(" - EASY : 단서에 [ ] 표시. 진행 힌트 무제한!\n");
    printf(" - NORMAL : 기호 숨김. 진행 힌트 3회 제한.\n");
    printf(" - HARD : 기호 숨김 & 진행 힌트 불가! 오직 직관으로 탈출.\n\n");

    // 3. 특수 명령어(편의 기능) 설명
    printf("3. [특수 명령어]\n");
    printf(" - 인벤토리 / 조합 / 분해 / 힌트\n");
    printf(" - 0 : 그냥 포기하기 / 9 : 현재 진행 상황 저장하고 돌아가기\n\n");

    // 4. 조작 팁 안내
    printf("4. 단말기 등 조작 도중 [ESC] 키로 물러날 수 있습니다.\n\n");

    // 💡 설명을 다 읽을 때까지 화면을 멈춰주는 도우미 함수 호출
    wait_for_esc();
}
```

파일 업그레이드 (23)

[게임 이벤트 소리]

```
// 🎵 1. 메뉴 이동음 (짧고 가벼운 소리)
// 주파수 500Hz로 0.04초(40ms) 동안 짧게 재생
void sound_navigate(void) { Beep(500, 40); }

// 🎵 2. 아이템 획득음 (기분 좋은 오름차순 멜로디)
// 도(523) - 미(659) - 솔(784) 음계를 연속으로 재생하여 띠-리-링~ 하는 효과를 줌
void sound_item(void) { Beep(523, 100); Beep(659, 100); Beep(784, 150); }

// 🎵 3. 성공 / 정답음 (높고 경쾌한 멜로디)
// 미(659) - 솔(784) - 높은 도(1046)를 재생하여 총총! 하고 풀리는 쾌감을 줌
void sound_success(void) { Beep(659, 100); Beep(784, 100); Beep(1046, 250); }

// 🎵 4. 에러 / 오답음 (낮게 떨어지는 소리)
// 300Hz에서 200Hz로 삐-빅! 하고 떨어지는 소리로 실패나 경고를 알림
void sound_error(void) { Beep(300, 150); Beep(200, 200); }

// 🎵 5. 일반 행동음 (단일 효과음)
// 주파수 800Hz로 0.06초 동안 띡! 하고 나는 소리로, 단순 조사나 메뉴 선택 시 사용
void sound_action(void) { Beep(800, 60); }
```

파일 업그레이드 (24)

[텍스트 색상 변경]

{ 아이템을 얻었을 때 }

```
set_color(C_ITEM); // 글자색을 노란색
```

{ 없는 행동을 하거나 틀렸을 때 }

```
set_color(C_ERROR); // 글자색을 빨간색
```

{ 문이 열리거나 게임을 클리어했을 때 }

```
set_color(C_SUCCESS); // 글자색을 초록색
```

파일 업그레이드 (25)

[폰트 변경과 텍스트 사이즈 조절]

```
void set_console_font(void) {
    CONSOLE_FONT_INFOEX cfi;
    cfi.cbSize = sizeof(CONSOLE_FONT_INFOEX);
    cfi.nFont = 0;

    // 1 사이즈 설정 부분
    cfi.dwFontSize.X = 0;  // 가로 크기 (0으로 두면 세로에 맞춰 자동 비율 조절)
    cfi.dwFontSize.Y = 24; // 세로 크기 (폰트 사이즈를 24로 아주 큼직하게 설정!)

    cfi.FontFamily = FF_DONTCARE;
    cfi.FontWeight = FW_NORMAL; // 폰트 굵기 (일반)

    // 2 폰트(글꼴) 설정 부분 ('굴림체')
    // 인코딩 에러(글자 깨짐)를 방지하기 위해 유니코드(Hex)로 직접 한 글자씩 입력
    cfi.FaceName[0] = (WCHAR)0xAD74; // '굴'
    cfi.FaceName[1] = (WCHAR)0xB9BC; // '림'
    cfi.FaceName[2] = (WCHAR)0xCC68; // '체'
    cfi.FaceName[3] = (WCHAR)0x0000; // 문자열의 끝을 알리는 NULL 값

    // 위에서 설정한 폰트 정보(cfi)를 현재 콘솔 창에 최종 적용!
    SetCurrentConsoleFontEx(GetStdHandle(STD_OUTPUT_HANDLE), FALSE, &cfi);
}
```

파일 업그레이드 (26)

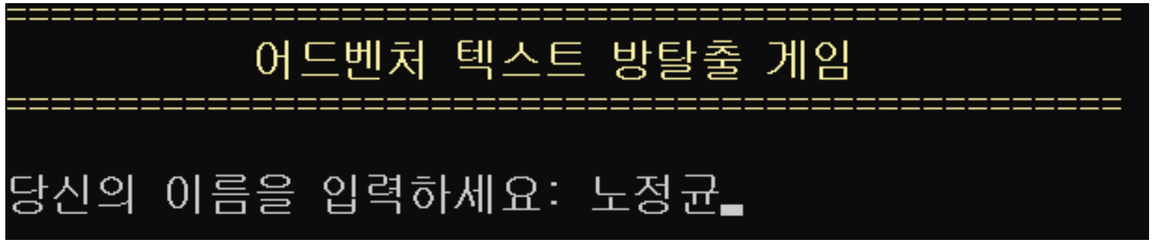
[콘솔 창 크기 조절]

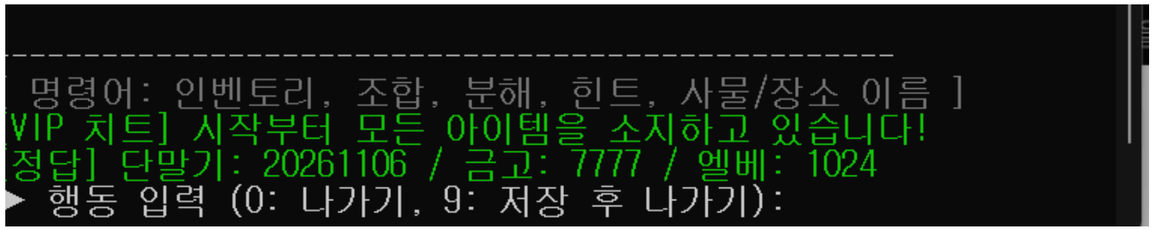
```
// 1. 콘솔 창의 '글자 칸 수'를 강제로 제한
system("mode con cols=65 lines=25");

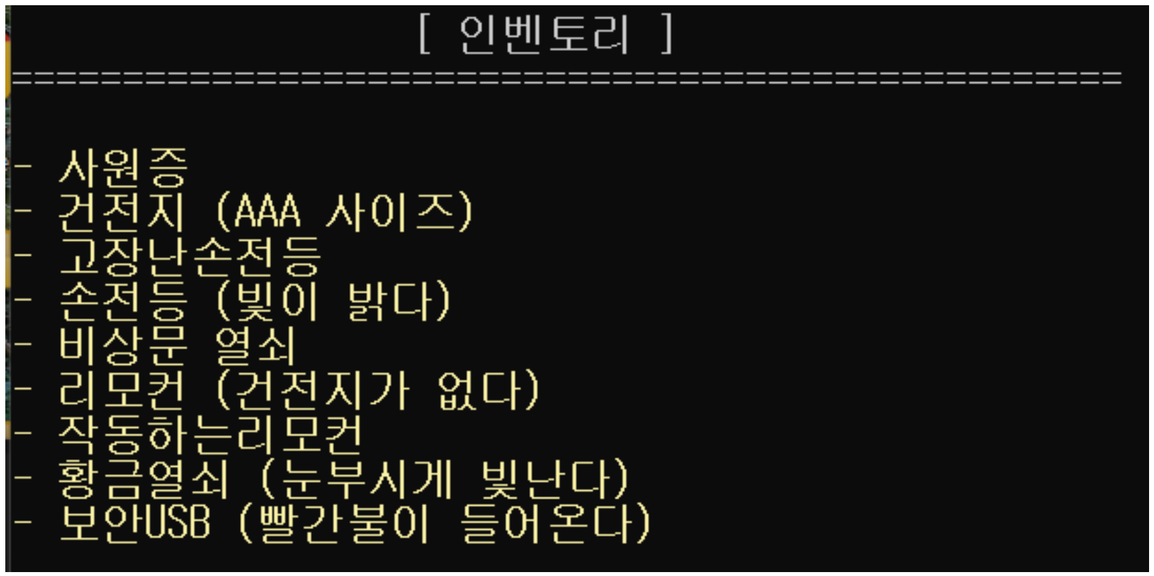
// 2. 현재 실행 중인 콘솔 창의 정보를 가져옴
HWND console = GetConsoleWindow();
RECT r;
GetWindowRect(console, &r);

// 3. 실제 모니터 상의 '창 크기(픽셀)'를 가로 780, 세로 600으로 강제 고정!
MoveWindow(console, r.left, r.top, 780, 600, TRUE);
```

파일 업그레이드 (27)







[이스터에그 추가]

```
else if (cheat_mode) {
    // 💡 치트 모드 ON: 에피소드 1의 모든 템 강제 지급!
    if (episode == 1) {
        has_id_card = 1;
        has_battery = 1;
        has_broken_flash = 1;
        has_flash = 1;
        has_key1 = 1;
        has_remote = 1;
        has_working_remote = 1;
        has_golden_key = 1;
        has_usb = 1;
    }
    // 💡 치트 모드 ON: 에피소드 2의 마스터키 강제 지급!
    else if (episode == 2) {
        has_masterkey = 1;
    }
}
```

총 업그레이드 내용

30 / 30

- 사용자 닉넴 입력과 활용
- HINT 모드 추가
- 메인 메뉴 선택(상하 키)
- HINT 모드 선택(상하 키)
- 무의미한 행동의 이벤트 처리
- 의미있는 행동의 이벤트 처리
- 반복된 의미있는 행동의 이벤트 처리
- 의미있는 행동의 파일 읽기 처리
- 흐름 이벤트 트리거
- 흐름 HINT 추가
- esc 입력으로 전 화면으로 돌아가는 로직
- 에피소드 해금 시스템 추가
- 분해 가능 이벤트 처리
- 분해 불가능 이벤트 처리
- 조합 가능 이벤트 처리
- 조합 불가능 이벤트 처리
- 인벤토리 확인 기능
- 특수한 이벤트 경우 인벤토리 확인 기능
- 난이도 설정 기능
- 소요시간 출력 기능
- 랭크 출력 기능
- 동적 클리어 씬 화면 출력
- 저장 및 이어하기 기능
- 게임 설명 옵션 기능 추가
- 게임 이벤트 소리
- 이벤트 종류별 텍스트 색상 변경
- 폰트 변경
- 텍스트 사이즈 변경
- 콘솔 창 크기 조절
- 치트모드 추가 [이스터에그]